הנחיות לפרויקט גמר – קורס Full Stack למתקדמים

מטרת פרויקט הגמר היא ליישם את כל הידע והכלים שנרכשו במהלך הקורס לכדי מוצר תוכנה שלם (End-to-End). על כל סטודנט או קבוצה לבחור רעיון למוצר פרקטי הניתן ליישום במסגרת הקורס.
לדוגמה: מערכת לניהול תורים, פלטפורמה לשיתוף תוכן לימודי, חנות E-commerce מתקדמת וכו'. אין לבחור ברעיונות מופשטים או תיאורטיים מדי; המערכת חייבת לפתור בעיה ממשית ולאפשר זרימת נתונים הגיונית.

- **הגשה:** ניתן להגיש את הפרויקט ביחידים, בזוגות, או בקבוצות של עד 4 סטודנטים.
- **היקף:** ככל שמספר הסטודנטים בקבוצה עולה, כך מצופה לראות מורכבות גבוהה יותר בכמות ה-Collections בבסיס הנתונים ובעושר הפונקציונלי של הממשק.

דרישות סף טכנולוגיות (חובה ליישום מתוך הסילבוס)

1. **צד שרת (Backend) – Node.js & Express**

- **REST API:** בניית שרת מאובטח עם CRUD מלא (יצירה, קריאה, עדכון ומחיקה).
- **Middleware:** שימוש ביישומי ביניים לניהול הלוגיקה והאבטחה בשרת.
- **Error Handling:** מימוש מנגנון גלובלי לטיפול בשגיאות.
- **Validation:** אימות נתונים נכנסים (בצד השרת) באמצעות ספריית **JOI**.

2. בסיס נתונים (Mongoose & MongoDB)

- **Schema Design:** הגדרת מודלים וסכימות עבודה מורכבות הכוללות קשרים בין Collections.
- **Mongoose:** חיבור מלא וביצוע שאילתות מתקדמות מול מסד הנתונים.

3. אבטחה ואימות משתמשים (Security & Authentication)

- **מערכת רישום:** בניית תהליך Login ו-Register מלא.
- **הצפנה:** שימוש ב-**bcrypt** להצפנת סיסמאות במסד הנתונים.
- **JWT:** אימות זהות המשתמש באמצעות JSON Web Tokens.
- **נתיבים מוגנים:** הגנה על נתיבי API (Protected Routes) ועל נתיבים בצד הלקוח.

4. Frontend (React SPA)

- **מבנה:** פיתוח אפליקציית Single Page Application מבוססת קומפוננטים.
- **Hooks:** שימוש ב-**useState** ו-**useEffect** לניהול מחזור החיים והמידע.
- **Axios:** ביצוע כל קריאות ה-API מול השרת (GET/POST/PUT/DELETE).
- **ולידציה:** מימוש React Form Validation בטפסים אינטראקטיביים.

5. ניהול State מתקדם (חובה להשתמש בשניהם)

- **Context API:** ניהול State גלובלי (למשל עבור פרטי המשתמש המחובר או הגדרות עיצוב).
- **REDUX:** ניהול מצב האפליקציה ב-Store מרכזי כולל Actions ו-Reducers (חיוני לחלקים המורכבים של האפליקציה).

6. ניהול מדיה ואופטימיזציה

- **File Upload:** שימוש ב-**Multer** להעלאת קבצים ותמונות ושמירתם ב-MongoDB.
- **הצגת מדיה:** הצגה דינמית של הקבצים ב-React.
- **ביצועים:** שימוש ב-Lazy Loading ו-**Memoization** (React.memo) לשיפור חוויית המשתמש.

7. Deployment (פרסום בענן)

- העלאת צד הלקוח ל-**Vercel** או **Netlify**.
- העלאת צד השרת ל-**Heroku**.
- הגשת הפרויקט כקישור חי (URL) שניתן לבחון בזמן אמת.

---

1. **קוד נקי:** הקוד צריך להיות קריא, מתועד ומחולק לתיקיות בצורה לוגית (Controllers, Models, Routes, Components).
2. **UX/UI:** יש להשקיע בחוויית משתמש הכוללת Loading States (אינדיקציה בזמן טעינת נתונים) וטיפול ידידותי בשגיאות בממשק. אין צורך בעיצוב מעל הממוצע.
3. **הצגת הפרויקט:** במפגש האחרון, על כל צוות להציג Live Demo של המערכת ולהסביר על הארכיטקטורה של הקוד (צד שרת וצד לקוח).

בהצלחה!